ARQUITECTURA DE DATOS

DevOps para arquitecturas de datos

Cultura, automatización y calidad a lo largo del ciclo de vida de los datos

AGENDA

Ruta de la sesión

01 Fundamentos de DevOps

02 DevOps aplicado a datos: DataOps y MLOps

03 CI/CD para pipelines de datos

04 Infraestructura como código

05 Calidad y observabilidad de datos

06 Herramientas y buenas prácticas

07 **Ejercicio práctico en Python**

FUNDAMENTOS · MODELO CALMS

¿Qué es DevOps?

Una cultura y un conjunto de prácticas que integran **desarrollo** y **operaciones** para entregar cambios de forma rápida, confiable y continua.

C

Cultura

Colaboración y responsabilidad compartida entre equipos.

A

Automatización

Eliminar tareas manuales y repetitivas.

L

Lean

Flujo continuo, lotes pequeños, menos desperdicio.

M

Medición

Decidir con métricas del proceso y del sistema.

S

Compartir

Conocimiento, herramientas y aprendizajes abiertos.

FUNDAMENTOS

El ciclo de vida DevOps



El monitoreo alimenta de nuevo la planificación: es un **ciclo continuo**, no una línea recta.

DEVOPS EN DATOS

¿Por qué DevOps en una arquitectura de datos?

Sin prácticas DevOps	Con prácticas DevOps
Despliegues manuales de pipelines	Despliegues automatizados y repetibles
El usuario final descubre los datos erróneos	Pruebas de calidad antes de llegar a producción
“En mi máquina sí funciona”	Ambientes idénticos con contenedores e IaC
Cambios de esquema sin control	Código, esquemas y configuración versionados en Git
Fallas silenciosas durante la noche	Monitoreo con alertas automáticas y responsables

DEVOPS EN DATOS

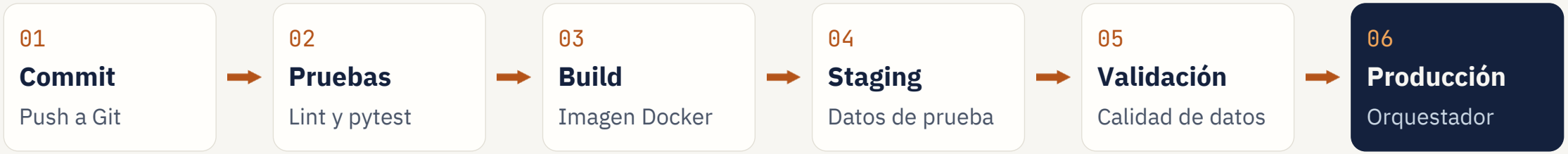
De DevOps a DataOps y MLOps

	DevOps	DataOps	MLOps
Foco	Aplicaciones	Pipelines y datos	Modelos de ML
Qué se prueba	Funcionalidad del código	Esquema, calidad y frescura	Desempeño y deriva del modelo
Artefacto	Binario o contenedor	Tablas, datasets, DAGs	Modelo entrenado
Métrica clave	Frecuencia de despliegue	Confiabilidad de los datos	Precisión en producción

Mismos principios —automatizar, probar, observar— aplicados a un artefacto distinto.

PRÁCTICAS CLAVE

CI/CD para pipelines de datos



CI · Integración continua

Cada cambio se integra a la rama principal y dispara pruebas automáticas de código y de datos.

CD · Entrega y despliegue continuos

Lo que pasa las pruebas se despliega automáticamente a staging y, con aprobación, a producción.

PRÁCTICAS CLAVE

Infraestructura como código (IaC)

Definir almacenamiento, bases de datos y redes en archivos versionados, en lugar de configurarlos a mano.

Reproducible: mismo código, mismo ambiente.

Revisable: cambios mediante pull request.

Recuperable: recrear todo en minutos.

Herramientas: Terraform, Pulumi, AWS CloudFormation, Docker Compose.

main.tf · Terraform

```
# Lago de datos para el curso
resource "aws_s3_bucket" "lago_datos" {
  bucket = "lago-datos-curso"
}

# Catálogo para consultas analíticas
resource "aws_glue_catalog_database" "analitica" {
  name = "analitica"
}
```

PRÁCTICAS CLAVE

Pruebas en tres capas

Pruebas unitarias

Verifican cada función de transformación con datos pequeños y controlados.

pytest

Pruebas de datos

Validan esquema, nulos, unicidad, rangos y reglas de negocio sobre los datos reales.

Great Expectations, dbt tests, Pandera

Pruebas de integración

Ejecutan el pipeline completo, de la fuente al destino, con una muestra representativa.

pytest + contenedores de prueba

Una prueba de datos que falla debe **detener el despliegue**, igual que una prueba de código.

PRÁCTICAS CLAVE

Los cinco pilares de la observabilidad de datos

01
Frescura
¿Llegaron los datos a tiempo?

02
Volumen
¿Llegó la cantidad de registros esperada?

03
Esquema
¿Cambiaron columnas o tipos?

04
Distribución
¿Los valores están en rangos normales?

05
Linaje
¿De dónde vienen y a quién afectan?

Métricas operativas útiles: duración de cada tarea, filas leídas frente a filas cargadas, tasa de fallos y tiempo de recuperación.

PRÁCTICAS CLAVE

Ecosistema de herramientas

Categoría	Herramientas representativas
Control de versiones	Git, GitHub, GitLab
CI/CD	GitHub Actions, GitLab CI, Jenkins
Orquestación	Apache Airflow, Dagster, Prefect
Transformación	dbt, Apache Spark, pandas
Contenedores	Docker, Kubernetes
Infraestructura como código	Terraform, Pulumi
Calidad de datos	Great Expectations, Soda, Pandera
Monitoreo y linaje	Prometheus, Grafana, OpenLineage

PRÁCTICAS CLAVE

Seis buenas prácticas

Todo en Git

Código, SQL, DAGs, esquemas y configuración bajo control de versiones.

Ambientes separados

Desarrollo, staging y producción, nunca probar directo en producción.

Pipelines idempotentes

Re-ejecutar un proceso no duplica ni corrompe los datos.

Cambios pequeños

Entregas frecuentes y fáciles de revisar y revertir.

Datos de prueba seguros

Muestras representativas y anonimizadas, sin datos personales reales.

Alertas con dueño

Cada alerta tiene un responsable y un procedimiento de respuesta.

EJERCICIO PRÁCTICO

Pipeline ETL con pruebas e integración continua

Construiremos en Python un pipeline de ventas que se prueba y se ejecuta automáticamente en cada cambio.

Python pandas pytest GitHub Actions

EJERCICIO PRÁCTICO

Enunciado del ejercicio

Contexto: una empresa recibe a diario un CSV de ventas con duplicados, montos vacíos y valores negativos.

1. Crea el archivo de datos con columnas id_venta, fecha y monto.
2. Implementa extraer, transformar y cargar en etl.py.
3. Escribe pruebas con pytest para las reglas de calidad.
4. Configura GitHub Actions para ejecutar pruebas y pipeline en cada push.

requirements.txt: pandas, pytest

Estructura del proyecto

```
pipeline-ventas/
├── data/ventas.csv
├── etl.py
├── tests/test_etl.py
├── requirements.txt
└── .github/workflows/ci.yml
```

data/ventas.csv

```
id_venta,fecha,monto
1,2026-01-05,100.5
2,2026-01-06,-20
2,2026-01-06,-20
3,2026-01-07,
4,2026-01-08,75
```

EJERCICIO PRÁCTICO · PASO 2

El pipeline: etl.py

```python
import sqlite3
import pandas as pd

def extraer(ruta):
    return pd.read_csv(ruta)

def transformar(df):
    df = df.dropna(subset=["id_venta", "monto"])
    df = df.drop_duplicates(subset="id_venta").copy()
    df["fecha"] = pd.to_datetime(df["fecha"])
    df["monto"] = df["monto"].astype(float)
    return df[df["monto"] > 0]

def cargar(df, ruta_db):
    with sqlite3.connect(ruta_db) as con:
        df.to_sql("ventas", con, if_exists="replace", index=False)

if __name__ == "__main__":
    cargar(transformar(extraer("data/ventas.csv")), "ventas.db")
```

Extraer

Lee el CSV crudo en un DataFrame.

Transformar

Quita nulos y duplicados, convierte tipos y descarta montos no positivos.

Cargar

Reemplaza la tabla ventas en SQLite: la carga es idempotente.

EJERCICIO PRÁCTICO · PASO 3

Las pruebas: tests/test_etl.py

```python
import pandas as pd
from etl import transformar

def datos_crudos():
    return pd.DataFrame({
        "id_venta": [1, 2, 2, None],
        "fecha": ["2026-01-05"] * 4,
        "monto": ["100.5", "-20", "-20", "50"],
    })

def test_sin_duplicados_ni_nulos():
    df = transformar(datos_crudos())
    assert df["id_venta"].is_unique
    assert df["id_venta"].notna().all()

def test_montos_positivos():
    df = transformar(datos_crudos())
    assert (df["monto"] > 0).all()
```

Cada regla de calidad se convierte en una prueba automática.

Ejecutar desde la raíz del proyecto:

```
$ python -m pytest -v
2 passed
```

Usar **python -m pytest** agrega la raíz al path, así la prueba puede importar etl.py.

EJERCICIO PRÁCTICO · PASO 4

Integración continua con GitHub Actions

.github/workflows/ci.yml

```yaml
name: CI pipeline de ventas
on: [push, pull_request]

jobs:
  pruebas:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: python -m pytest -v
      - run: python etl.py
```

En cada push o pull request:

1. GitHub crea una máquina limpia con Ubuntu y Python 3.12.
2. Instala las dependencias declaradas en requirements.txt.
3. Ejecuta las pruebas: si una falla, el flujo se detiene en rojo.
4. Si todo pasa, corre el pipeline completo con los datos de ejemplo.

EJERCICIO PRÁCTICO

Retos de extensión y entregables

Retos

1. Rechazar ventas con fecha futura y agregar su prueba.
2. Registrar en un log las filas leídas frente a las cargadas.
3. Empaquetar el pipeline en un Dockerfile.
4. Romper una prueba a propósito, ver el CI en rojo y corregirlo.

Entregables

- Enlace al repositorio en GitHub.
- Captura del workflow en verde.
- Informe de una página con las reglas de calidad y decisiones tomadas.

Fecha de entrega: Indicada en Canvas]

Rúbrica: funcionamiento del pipeline 30 % · pruebas 30 % · CI 25 % · documentación 15 %

CIERRE

DevOps convierte los pipelines de datos en productos confiables

Automatiza

Infraestructura, pruebas y despliegues como código versionado.

Prueba

El código y también los datos, antes de llegar a producción.

Observa

Frescura, volumen, esquema, distribución y linaje.

¿Preguntas?